Lecture 10 · Part II · Lecture notes

PyTorch

Dataset: Fashion-MNIST

How to use these notes

The primary text for this lecture is Géron, Chapter 10. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 Where we are **2**

2 Derivation: backpropagation as reverse-mode autodiff **2**

2.1 Two bracketings, two algorithms 2

2.2 A worked example small enough to check 3

2.3 The argument that settles it 3

3 What the wrapper costs, measured **4**

3.1 So when does the accelerator pay? 4

4 Tensors, autograd, and the loop **4**

5 Two bugs that never raise **5**

5.1 The missing `zero_grad()` 5

5.2 The missing `model.eval()` 6

6 Feeding the loop, and one more silent error **6**

6.1 Searching the hyperparameters properly 7

6.2 Saving the model, and checking the reload 7

7 Where we ended **8**

8 The notebook **8**

9 Exercises **8**

Summary **9**

---

1 Where we are

Lecture 9 found four things `MLPClassifier` will not let you do: change what it optimises, see a gradient, stop mid-epoch, or move the computation to another device. All four are one wall — `fit()` contains a loop over epochs and batches that you did not write, cannot see and cannot enter.

The repair is not a better library. It is *owning the loop*. Today: what that loop has to compute, why it can be computed at all, and how to write it.

2 Derivation: backpropagation as reverse-mode autodiff

You called `clf.fit()`. It computed 266,610 partial derivatives, on every batch, twenty times over. *How?* The cost of that computation is what decides whether training a network is possible at all — and the reason it is possible is a property of the *direction* in which the chain rule is applied.

2.1 Two bracketings, two algorithms

A network is a composition, so writing $L = f_3(f_2(f_1(\boldsymbol{\theta})))$ gives

$$\frac{\partial L}{\partial \boldsymbol{\theta}} = J_3 J_2 J_1,$$

with J_i the Jacobian of f_i evaluated at the point the forward pass reached. Every layer contributes one factor — and matrix multiplication is associative:

$$\underbrace{(J_3 J_2) J_1}_{\text{reverse mode}} \;=\; \underbrace{J_3 (J_2 J_1)}_{\text{forward mode}} .$$

Same number. Wildly different cost.

Forward mode picks a direction $\mathbf{v}$ in input space and carries a tangent $\dot{u} = \partial u / \partial v$ alongside every intermediate value. One sweep gives the derivative of every output with respect to *one* input direction, and nothing needs to be stored, because the tangent travels with the value.

Reverse mode picks a direction in output space — for a scalar loss there is only one, and it is 1 — and carries an adjoint $\bar{u} = \partial L/\partial u$ backwards. One sweep gives the derivative of *one* output with respect to every input, and the forward values must be kept, because the Jacobians are evaluated at them.

Mode	Sweeps needed	Cheap when
Forward	n — one per input	few inputs, many outputs
Reverse	m — one per output	many inputs, few outputs

2.2 A worked example small enough to check

$$L = w^2, \qquad w = xy + \sin x, \qquad \text{at } x = 2,\ y = 3.$$

By hand, $\partial L/\partial x = 2w(y + \cos x)$ and $\partial L/\partial y = 2wx$.

Node	Expression	Value	Adjoint	Computed as	Value
x	input	2.0	$\partial L/\partial L$	the seed	1
y	input	3.0	$\partial L/\partial w$	$2w$	13.8186
u	$x \cdot y$	6.0000	$\partial L/\partial u$	$(\partial L/\partial w) \cdot 1$	13.8186
s	$\sin x$	0.9093	$\partial L/\partial s$	$(\partial L/\partial w) \cdot 1$	13.8186
w	$u + s$	6.9093	$\partial L/\partial y$	$(\partial L/\partial u)\, x$	27.6372
L	w^2	47.7384	$\partial L/\partial x$	$(\partial L/\partial u)\, y + (\partial L/\partial s) \cos x$	35.7052

Six nodes evaluated once each in topological order, then six lines backwards — one pass, both derivatives. Forward mode would need two sweeps here, one seeded at x and one at y.

The only bookkeeping in the whole method

x is used twice, once by the product and once by the sine, so

$$\frac{\partial L}{\partial x} = \underbrace{\frac{\partial L}{\partial u}\, y}_{\text{via } \times} + \underbrace{\frac{\partial L}{\partial s} \cos x}_{\text{via sin}}.$$

A node's adjoint is the *sum* of the contributions along its outgoing edges. Miss one edge and the gradient is silently wrong, not absent.
Reverse-mode autodiff is a topological sort, a table of local derivatives, and this sum. There is no other idea in it.

The notebook checks this against `torch`: autograd returns the same two numbers as the hand derivatives, to machine precision.

2.3 The argument that settles it

Training differentiates a function from 266,610 parameters to *one* scalar loss. Forward mode needs one sweep per input: 266,610 of them. Reverse mode needs one per output: 1.

Reverse mode is not a good choice

It is the *only viable* one, by a factor equal to the parameter count.

So what is backpropagation?

Reverse-mode automatic differentiation, applied to a neural network, with the seed set to 1 at the loss. It is not specific to neural networks, it is not an approximation, and it is not gradient descent — it supplies the gradient that descent then uses. The 1986 paper's contribution was noticing this was cheap enough to be practical, not inventing the chain rule.

3 What the wrapper costs, measured

The accuracy was not the bug. 87.28% validation accuracy against a 10.0% baseline is a real result: the split was clean, the validation set separate, the test set touched once. The bug is that you cannot do anything else.

10 epochs on 12,000 images, training loop only	Wall clock	Validation accuracy
Scikit-Learn, CPU	2.1 s	87.28%
PyTorch, CPU	1.4 s	86.36%
PyTorch, MPS	0.8 s	86.52%

Read the rows in pairs. Rows one and two are the same CPU, so their ratio is the framework; rows two and three are the same code, so their ratio is the device. *At this size neither is large: both are under 2×.* Timing PyTorch on the accelerator against Scikit-Learn on the CPU would have given one number for two changes.

3.1 So when does the accelerator pay?

A forward pass is $\phi(\mathbf{XW} + \mathbf{b})$ repeated, and a GPU is a machine for large matrix products and very little else. Ours are not large: 266,610 parameters and a batch of 128 is a small matrix product, so much of each step is the cost of dispatching it rather than the arithmetic.

Widen the layers and measure again, with identical code on both devices. At the smallest width the two devices nearly tie; from there the ratio climbs, to nearly four at the widest, because each batch carries more arithmetic to cover the cost of dispatching it. Measured with 2 epochs on 12,000 images, batch 128, identical code on both devices, on an Apple MPS backend; a Colab T4 has a different crossover, in the same place in the argument.

4 Tensors, autograd, and the loop

A tensor is an array that also knows two things a NumPy array does not: where it lives — an operation between tensors on different devices is an error, and a loud one — and whether to record.

`requires_grad` does not compute a derivative. It says *record what happens to me*. Every operation

on a recording tensor creates a node holding the operation and its inputs; the graph is built during the forward pass, by running it, with no separate compilation step; and it is rebuilt from scratch on every batch, which is why a Python `if` inside `forward` just works. That is the forward sweep from §2, with the graph written down as it goes so the reverse sweep has something to walk — and you can inspect it: `r.grad_fn` and `r.grad_fn.next_functions` are the nodes of the worked example, by name.

Evaluation needs no derivatives, so `torch.no_grad()` stops the recording and saves memory proportional to the batch. Forgetting it is slow and memory-hungry, but it is not a correctness bug. The two that follow are.

```
for epoch in range(EPOCHS):
    model.train()
    for xb, yb in train_loader:
        opt.zero_grad()          # 1  clears p.grad, which backward() adds to
        out  = model(xb)         # 2  the forward sweep; the graph is recorded
        loss = lossf(out, yb)    # 3  reduces the batch to one scalar
        loss.backward()          # 4  the reverse sweep; fills p.grad
        opt.step()               # 5  updates the parameters from p.grad
```

Five lines. There is no hidden machinery: this is the whole of what `fit()` was doing — and every arrow is a line you wrote, so every arrow is a place you can print, clip, log, branch or stop.

Rebuilt in PyTorch and run on MPS, the model reaches 86.52% validation accuracy against Scikit-Learn's 87.28%. The accuracies agree, as they must: same architecture, same optimiser, same learning rate, same epochs, same data. *The rebuild is not an improvement — it is the same model.* What we bought is the loop. Overlaying the two training-loss curves is the first check after any rewrite; if they had disagreed, one of the two implementations would be wrong.

5 Two bugs that never raise

5.1 The missing `zero_grad()`

`backward()` *accumulates*: `p.grad` is added to, not overwritten, so leaving out the line that clears it is not an error.

Failure condition — what it does to training

At step t the optimiser sees the sum of every gradient computed so far, not the current one. The effective step size grows without bound, and Adam's normalisation hides the divergence for a while. No exception, no warning, no `NaN`.

	Final training loss	Validation accuracy
With `opt.zero_grad()`	0.3101	86.52%
Without it	1.9989	25.42%

61.10 points: 10 epochs on 12,000 images, identical seeds, identical everything else.

5.2 The missing `model.eval()`

Some layers behave differently in the two phases. Dropout zeroes a random fraction of units in `train()` and passes everything through in `eval()`; batch normalisation uses this batch's statistics in one and the running averages in the other. *The module defaults to training mode*, so building a model and never calling `eval()` means every evaluation you make is of a randomly crippled network. Batch normalisation arrives in Lecture 11; the same failure applies to it and is harder to see, because it does not fluctuate.

Evaluation mode	**Test accuracy**
`model.eval()`	85.69%
`model.train()`, mean of 10 calls	84.98%
lowest of those 10	84.77%
highest of those 10	85.22%

Dropout 0.2, measured on 10,000 test images: the missing call costs 0.71 points.

The tell is the spread, not the mean

0.45 accuracy points across ten *identical* calls. A deterministic function of fixed weights and fixed data returns the same number every time, so if your metric wobbles between calls, ask which layer is still in training mode.
Run any evaluation twice. It costs one line, and it catches this bug, the seeding bugs, and any accidental shuffling of the labels.

Both bugs are one line, neither raises, neither warns, and both produce a plausible number — costing 61.10 points and 0.71 points respectively. One is caught by reading the loop, the other by running it twice, and both survive a review that is looking for exceptions rather than for behaviour.

6 Feeding the loop, and one more silent error

`Dataset` answers two questions — how many, and give me number i — and `DataLoader` shuffles, batches, and can load in parallel while the device is busy. Indexing a big tensor by hand works while the data fits in memory; it stops working once the data does not, and the interface does not change.

Failure condition — the last batch is short

10,000 test images in batches of 384 gives 27 batches, the last containing 16 — so a plain mean of per-batch accuracies weights those 16 exactly as heavily as a full batch.

How the accuracy was aggregated	Value
Counted over the whole set	85.600%
Mean of the per-batch accuracies	85.667%

0.067 points, and this time the per-batch mean is the *higher* of the two — small *here*, because the last batch is 16 of 384 and the model is not much better or worse on it; the sign says only that those 16 went better than the rest. Change either and it grows: with batch size 3,000 on 10,000 images the short batch carries a quarter of the weight instead of a tenth. `drop_last=False` is the default and that is the right default. The bug is in how the metric is aggregated, not in the loader.

Weight by the batch size, or count over the set — never take a plain mean of per-batch metrics. Not because the error is always large, but because *you cannot tell from the number whether it was*.

A custom module subclasses `nn.Module` and writes `forward` itself. One rule: `super().__init__()` before anything else. It creates the registry the layers and parameters are recorded in, so assigning a layer before it fails at once with `AttributeError: cannot assign module before Module.__init__() call`; everything after that — `.to(device)`, `.parameters()`, `state_dict()` — comes for free.

6.1 Searching the hyperparameters properly

Lecture 9 tuned by hand: ten points, one axis at a time. **Optuna** proposes a configuration, trains it briefly, scores it, and lets a sampler choose the next from what it has learned — here a learning rate on a log scale between 10^{-4} and 10^{-2} and a dropout rate in steps of 0.1. It is a search over configurations, not a final model. Not examinable: Optuna is not in the book. It is here because it is what you will actually use, and because it makes the point that hand tuning was never a search.

6.2 Saving the model, and checking the reload

```
torch.save(model.state_dict(), "sorter.pt")      # the tensors
fresh = build_model().to(device)
fresh.load_state_dict(torch.load("sorter.pt", weights_only=True))
```

The tensors are 1041 KB, 266,610 float32 numbers; the file weighs a little more, because `state_dict` is written as a zip archive with the names and the container inside it. Do not `torch.save(model, ...)`: that pickles the object, and unpickling needs your class definition at the same import path, so next term, when the file has moved, the checkpoint will not load.

Then check the reload, with an exact equality:

```
assert accuracy(model, X_test, y_test) == accuracy(fresh, X_test, y_test)
```

Here `accuracy` counts hits over the whole set, in `eval()` mode, under `no_grad`. Not "it looks about the same": a reloaded network is either the same function or it is not. The usual cause of failure

is loading a `state_dict` into a model built with different widths. PyTorch raises for that — but only if you did not pass `strict=False` to silence it.

7 Where we ended

Model	Validation accuracy	Wall clock
Always one class	10.0%	—
Scikit-Learn MLP, CPU	87.28%	2.1 s
PyTorch, same model, MPS	86.52%	0.8 s
The same code without `zero_grad()`	25.42%	unchanged

The last row is the one to remember: it ran, it finished, and it reported a number. The PyTorch model's test accuracy, touched once, is 85.60%.

Silent failure	Diagnosed in	Measured cost
Scaler fitted before the split	Part I	nothing measurable
Evaluating on training data	Part I	an RMSE of zero
Accuracy under class imbalance	Part I	a useless model above 90%
Hyperparameters chosen on the test set	Part I	an optimistic report
Missing `optimizer.zero_grad()`	Lecture 10	61.10 points
Missing `model.eval()`	Lecture 10	0.71 points
Metric averaged per batch	Lecture 10	0.067 points

Every one of them ran to completion and printed a plausible number.

8 The notebook

What to change

Make the network four times deeper and re-run the training loop, as the notebook's closing cell suggests. It trains far worse than its size suggests it should — and Lecture 11 derives why, and repairs it.
Either device is fine: everything is sized for the free CPU tier, and the measurement in §3 shows what an accelerator buys at this size — less than a factor of two.

9 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 11's deck and in the exercise book.

1. Reverse-mode automatic differentiation costs one sweep for all partial derivatives; forward mode costs one per input. State the condition on the function's shape that makes reverse mode the right choice. [4]

2. Test accuracy on 10,000 images is computed as the plain mean of the per-batch accuracies, in batches of 384. State why this is wrong, and the rule that fixes it. [3]

3. The five lines of the training loop are reordered so that `opt.step()` comes before `loss.backward()`. State what the student observes. [4]

4. A network with dropout is evaluated without calling `model.eval()`. Describe the symptom precisely, and the one-line diagnostic that catches it. [5]

5. In the measured comparison the accelerator barely beat the CPU on the smallest network, and its lead grew as the layers widened. Give the reason, and the quantity that decides where the crossover sits. [4]

Summary

The chain rule is a product of Jacobians and the product may be bracketed from either end. Forward mode costs one sweep per input; reverse mode costs one sweep per output; training has 266,610 inputs and one scalar output, so reverse mode is the only viable choice — by a factor equal to the parameter count. Backpropagation is that reverse sweep, seeded with 1 at the loss.

Moving to PyTorch and moving to the accelerator each bought under a factor of two at this size, measured separately; the accelerator's lead grows with the arithmetic each batch carries, from near-parity at the smallest width to nearly four times at the widest. The rebuilt model is the *same* model, and what was bought is the five-line loop.

Then three bugs that raise nothing, warn nothing, and return a plausible number. A missing `zero_grad()` costs 61.10 accuracy points because `backward()` accumulates by design; a missing `model.eval()` costs 0.71, and its tell is a 0.45-point spread across ten identical calls; a plain mean of per-batch accuracies is off by 0.067 points here, by an amount the number itself cannot reveal.